

Evaluating AI-Generated Text Detection:

A Comparative Analysis of DetectGPT, Retrieval-Based Defenses, and Alternative Solutions

Davide Pietragalla

Department of Computer Science

Sapienza University of Rome

Rome, Italy

pietragalla.1965270@studenti.uniroma1.it

Kevin Huang

Department of Computer Science

Sapienza University of Rome

Rome, Italy

huang.1993737@studenti.uniroma1.it

Abstract—The increasing fluency and widespread usage of large language models (LLMs) have been accompanied by a rise in malicious applications, such as fake content creation and academic plagiarism. This trend highlights the need for tools that aid in the detection of LLM-generated text, including approaches based on watermarking or outlier detection. We will evaluate state-of-the-art approaches to AI-generated text detection, such as DetectGPT [1], and its robustness when subjected to stress from parameter paraphrase generation model (DIPPER [2]). Our object is to enhance the robustness of the detection, integrating retrieval-based defenses.

I. INTRODUCTION

(Scope of the Review)

In recent years, the rise of LLMs have brought to a surge in malicious applications, notably the generation of fraudulent content and academic plagiarism. Since these models mimic human-like reasoning, the incessant growth of such adversarial misuses represents an escalating threat to the integrity of digital communication.

In this report we’re going to dive in an analysis of different detection techniques: starting of

II. TAXONOMY OF DETECTION METHODS

In this section, we provide a comprehensive overview of existing approaches for detecting AI-generated text.

A. Classifier-Based Methods

As the reader may know, LLM-generated text detection aims to distinguish whether a piece of text was produced by an LLM, which is essentially a binary classification task.

1) *Features-Based Classifiers*: Features-based classifiers can focus on language features or model features.

For what concerns language features-based classifiers, the process typically starts with the extraction of key statistical language features, followed by the application of machine learning techniques to train a classification model. These features can be extensively categorized into stylistic features, complexity features, semantic features, psychological features, and knowledge-based features, and they are primarily obtained through statistical methods.

An interesting example of this methodology is provided by Shah et al. [3] (2023), who developed a classifier leveraging an ensemble of machine learning models, including logistic regression, decision trees, random forests, support vector

classifiers, and gradient boosting. By focusing on a variety of stylistic features, their research demonstrated impressive efficacy, achieving an accuracy of up to 93% in distinguishing between AI-generated and human-written text.

A further example is the one proposed by Mindner et al. [4] (2023), who employed a multidimensional framework to enhance the classifier’s discriminative power. Their methodology leverages a wide list of features, including complexity metrics, semantic analysis, error-based features, list searches, readability assessments, alongside artificial intelligence feedback and text vectorization. This optimized detector proved to be highly effective, ultimately outperforming GPTZero by a substantial 183.8% in F1 Score, showing its superior precision of multifaceted detection systems over baseline tools.

Although classifiers based on linguistic features have demonstrated significant efficacy, their shortcomings cannot be overlooked. The results from Shah and Mindner indicate that such feature classifiers have poor robustness against ambiguous semantics and often underperform compared to neural networks. Moreover, although classifiers based on stylistic features are capable of distinguishing between texts written by humans and those generated by LLMs, their ability to detect misinformation generated by LLMs is actually limited. This limitation is also highlighted by Schuster et al. [5] (2020), which proves that language models tend to produce stylistically coherent texts.

The branch of model features-based classifiers is not only capable of detecting texts generated by LLMs but can also be employed for text provenance tracing.

One interesting approach, proposed by Wang et al. [6] (2023) involves utilizing log-probability lists extracted from white-box LLMs as features for AI-generated text (AIGT) detection. The authors assert that these probability sequences are analogous to waves in speech processing, which standard LLMs are not capable of analyzing effectively. Therefore, they designed a context network that combines a CNN with a two-layer Transformer. This architecture encodes the text by processing these sequences and identifies AI-generated content through a sequence tagging mechanism.

Another approach is the one introduced by Wu et al. [7] (2023), which involves a multi-task learning framework designed to simultaneously address text detection and authorship attribution. This method leverages a combination of features

such as log likelihood, log rank, entropy, and LLM bias to train a neural network classifier, which ultimately achieved an average F1 score of 98.41%.

However, a significant drawback of these methods is that they all require access to the source model’s logits. Therefore, for powerful closed-source models where logits are inaccessible, these methods remain ineffective.

2) *Pre-Training Classifiers*: These methods involve fine-tuning Transformer-based LMs, applying contrastive learning, or employing adversarial learning.

The first approach involves fine-tuning the model on a specific domain. Although recent studies, such as Guo et al. (2023), Liu et al. (2023), Chen et al. (2023), and Wang et al. (2023), achieve average accuracies of approximately 95%, these models are prone to overfitting. [8]–[11]

The second approach is primarily used to address data scarcity. Since it works by learning a representation space where similar AI-generated texts are clustered together, these models remain robust against unseen generative models.

Bhattacharjee et al. (2023) [12] proposed ConDA, a contrastive domain adaptation framework, which combines standard domain adaptation technology with the representation capabilities of contrastive learning, greatly improving the model’s defense capabilities against unknown models.

The final approach focuses on making detectors less vulnerable to attacks such as paraphrasing. Common techniques include sample enhancement, where AI-generated texts are paraphrased to train the detector to recognize camouflaged content, and two-player games, in which an attacking paraphraser and a detector learn iteratively from one another to enhance performance.

A notable framework based on two-player games is OUTFOX, proposed by Koike et al. (2023) [13]. In this framework, the attacker utilizes the detector’s predicted labels as In-Context Learning (ICL) exemplars to generate text that poses significant detection challenges. Conversely, the detector treats this adversarially generated content as ICL exemplars to enhance its ability to identify sophisticated attackers. This reciprocal exchange of outputs creates a feedback loop that substantially improves the robustness of detectors against LLM-generated text.

3) *LLMs-Based Detectors*: Several works have examined the feasibility of utilizing LLMs as detectors to discern text generated by either themselves or other LLMs.

Empirical evidence suggests that the reliability of LLMs in detecting synthetic text is inversely proportional to their complexity. While earlier models like Grover demonstrated effective self-detection, more complex models such as GPT-4 fail significantly in zero-shot environments. Interestingly, these models exhibit contrasting biases: ChatGPT tends toward leniency, misclassifying AI-generated text as human in 50% of cases, whereas GPT-4 is overzealous, erroneously labeling 95% of human-written content as AI-generated. [14]

The transition from ‘native’ detection to In-Context Learning (ICL) marks a critical methodological shift. Frameworks like OUTFOX demonstrate that the core issue is not a lack of

comprehension, but rather a lack of reference. By providing exemplars within the prompt, LLMs can overcome intrinsic limitations and even outperform specialized RoBERTa-based detectors. Ultimately, combining ICL with two-player adversarial dynamics suggests that effective detection requires a ‘learning-to-learn’ system capable of evolving alongside increasingly sophisticated generative models. [15]

B. Statistics-Based Methods

Within the statistics-based framework, detection methods leverage the inherent probabilistic signatures of language models to identify machine-generated text without the need for supervised training. Unlike approaches that rely on manual feature engineering, these methods focus on the statistical regularities of token distributions, such as log-likelihood, rank, and entropy. By analyzing the “white-box” output probabilities of a model or the “black-box” curvature of the probability space, these techniques discern the subtle discrepancies between the predictive patterns of LLMs and the more irregular nature of human writing.

1) *White-Box Statistics*: White-box methods for detecting text generated by LLMs require direct access to the source model for implementation. The existing white-box detection techniques primarily use zero-shot approaches, which involves obtaining the model’s logits output and calculating specific metrics. These metrics are then compared against predetermined thresholds obtained through statistical methods to identify LLM-generated text.

Some white-box statistical approaches detect LLM-generated text by comparing the differences in performance metrics after statistical perturbation. Mitchell et al. [1] (2023) proposed a method to identify text produced by LLMs by analyzing structural patterns in the LLMs’ probability functions, specifically in regions of negative curvature. The premise is that LLM-generated text tends to cluster at local log-probability maxima. Detection involves comparing log-probabilities of text against those from the target LLM, using a pre-trained mask-filling model like T5 to create semantically similar text perturbations. While innovative and sometimes more effective than supervised methods, DetectGPT has limitations, including potential performance drops if rewrites don’t adequately represent the space of meaningful alternatives, and high computational demands, as it needs to score many text perturbations. [15]

Bao et al. [16] (2023) reported a method that replaces the perturbation step of DetectGPT with a more efficient sampling step, significantly improving the detection accuracy by about 75% and increasing the detection speed by 340 times.

In contrast to DetectGPT, the white-box configuration in DNA-GPT by Yang et al. [17] (2023) utilizes large language models such as ChatGPT to continue writing truncated texts instead of employing perturbation settings. It analyzes the differences between the original text and the continued text by calculating probability divergence, achieving a detection performance close to 100%.

DetectLLM by Su et al. [18] (2023) is another recent contribution, that parallels the conceptual framework of DetectGPT. It employs normalized perturbed log-rank for text detection generated by LLMs, asserting a lower susceptibility to the perturbation model and the number of perturbations compared to DetectGPT.

2) *Black-Box Statistics*: Unlike white-box statistical methods, black-box statistical methods utilize a black-box model to calculate specific feature scores of a text without needing access to the logits of the source or surrogate model.

One example of this approach is the one provided by Mao et al. [19] (2024) which identified LLM generated text by computing the similarity scores between the original texts and their rewritten and revised versions. These approaches are based on the observation that human-written texts tend to trigger more revisions when LLMs are tasked with rewriting and editing than LLM-generated texts.

However, the approaches of black-box statistics are not without challenges, including the substantial overhead of accessing the LLM and long response times. [15]

C. Watermarking

A watermark is a modification to the generated text that can be detected post-hoc by an algorithm while remaining imperceptible to human readers. Effective watermarks are difficult to remove and have little effect on the quality of generated text. [20]

1) *Data-Driven Watermarking*: These methods typically rely on a sort of backdoor insertion, where a small number of watermarked samples are included in the dataset in order to make the model learn a secret function set by the defender. When a specific trigger is activated, the backdoor watermark is triggered, which is usually implemented in a black-box setting. This solution protects the model from unauthorized fine-tuning or use beyond the terms of the license. Even if the model is fine-tuned for several downstream tasks, the watermark remains difficult to eradicate. [21]

It is important to note that data-driven methods were originally thought to protect copyright of datasets, and therefore, they usually lack substantial payload capacity and generalizability. Furthermore, applying these approaches in the field of LLM-generated text detection requires significant resource investment, such as the embedding of watermarks in a very large amount of data and the retraining of LLMs. [15]

2) *Model-Driven Watermarking*: Model-driven methods integrate watermarks by modulating the logits output distribution or token-sampling parts during the inference process. Consequently, the LLMs return answers that inherently carry these embedded watermarks, allowing for their subsequent identification. [15]

This specific branch is a sort of plug-and-play solution that offers a functional and reliable way to protect models without impacting text quality or adjusting parameters.

Two central approaches in this category are those proposed by Kirchenbauer et al. [22] and Liu et al. [23] (both 2023). The former pioneered a logits-based framework that biases the

sampling process toward a randomly selected set of 'green' tokens. While effective and minimally invasive to text quality, this method's dependence on a symmetric secret key introduces vulnerabilities, as key exposure can lead to watermark removal or forgery. To enhance robustness and security, Liu et al. advanced the field with two significant improvements. First, they proposed a semantic invariant method that utilizes semantic embeddings of preceding tokens to determine watermark logic, ensuring the watermark remains detectable even after synonym substitution or paraphrasing. Second, they addressed the security flaws of traditional keys by developing a private watermarking algorithm. Such a system employs two different neural networks for generation and detection, sharing specific parameters to optimize efficiency while eliminating the need for a vulnerable secret key.

Another noteworthy case is SemStamp by Hou et al. (2023), which is a robust sentence-level semantic watermarking algorithm that utilizes Locality-Sensitive Hashing (LSH). Unlike previous methods, SemStamp operates by partitioning the semantic embedding space into watermarked and non-watermarked regions. During the generation, this system repeatedly tests candidate sentences until the chosen output falls within a designated "marked" region of the embedding space. This approach has proven to be more resilient against paraphrase attacks. [24]

3) *Post-Processing Watermarking*: This last alternative of watermarking refers to a technique that involves embedding a watermark by processing the text after it has been output by a LLM. This solution functions as a separate module that is included in a pipeline with the output of the generative model.

This approach may have significant and various applications depending on specific cases. For instance, the proprietary and closed-source architecture of models such as GPT-5 makes internal watermarking nearly impossible. Nevertheless, post-processing watermarking offers a viable alternative by allowing third parties to independently embed watermarks into the outputs of black-box LLMs. [15]

III. VULNERABILITIES AND EVASION TACTICS: THE ROBUSTNESS GAP

A. Paraphrasing Attacks

B. Impact on Detection Accuracy

C. Text Mixing and Shortening

IV. RETRIEVAL-BASED DEFENSES: A SEMANTIC PARADIGM

A. The Database Approach

B. Semantic Search and Matching

C. Scalability and Efficacy

V. COMPARATIVE EVALUATION AND PRACTICAL LIMITATIONS

A. The Metric Problem (AUROC vs. FPR)

Discuti l'importanza di valutare questi sistemi mantenendo un False Positive Rate (FPR) molto basso (es. 1%), poiché

accusare falsamente un umano di aver usato l'AI ha gravi conseguenze.

VI. CONCLUSION AND FUTURE DIRECTIONS

(Synthesis of the Current Landscape)
(Future Research)

CONTRIBUTION STATEMENT

Each member reviewed and refined all sections of this document written by the other members to ensure accuracy and reduce redundancy. Specifically, each member was responsible for the following parts:

- Davide Pietragalla: Governance Background of Curve Finance, Core Mechanism Design of Curve Finance, Notable Incidents & Post-Mortems.

REFERENCES

- [1] E. M. Y. L. A. K. C. D. Manning and C. Finn, "Detectgpt: zero-shot machine-generated text detection using probability curvature," *ArXiv*, 2023.
- [2] J. W. Kalpesh Krishna Yixiao Song Marzena Karpinska and M. Iyyer, "Paraphrasing evades detectors of ai-generated text, but retrieval is an effective defense," *ArXiv*, 2023.
- [3] A. Shah, P. Ranka, U. Dedhia, S. Prasad, S. Muni, and K. Bhowmick, "Detecting and unmasking ai-generated texts through explainable artificial intelligence using stylistic features," *International Journal of Advanced Computer Science and Applications*, vol. 14, no. 10, 2023. [Online]. Available: <http://dx.doi.org/10.14569/IJACSA.2023.01410110>
- [4] L. Mindner, T. Schlippe, and K. Schaaff, "Classification of human- and ai-generated texts: Investigating features for chatgpt," *ArXiv*, vol. abs/2308.05341, 2023. [Online]. Available: <https://api.semanticscholar.org/CorpusID:260775600>
- [5] T. Schuster, R. Schuster, D. J. Shah, and R. Barzilay, "The limitations of stylometry for detecting machine-generated fake news," *Computational Linguistics*, vol. 46, no. 2, pp. 499–510, Jun. 2020. [Online]. Available: <https://aclanthology.org/2020.cl-2.8/>
- [6] P. Wang, L. Li, K. Ren, B. Jiang, D. Zhang, and X. Qiu, "SeqXGPT: Sentence-level AI-generated text detection," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, H. Bouamor, J. Pino, and K. Bali, Eds. Singapore: Association for Computational Linguistics, Dec. 2023, pp. 1144–1156. [Online]. Available: <https://aclanthology.org/2023.emnlp-main.73/>
- [7] X. Yu, K. Chen, Q. Yang, W. Zhang, and N. Yu, "Text fluoroscopy: Detecting LLM-generated text through intrinsic features," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, Y. Al-Onaizan, M. Bansal, and Y.-N. Chen, Eds. Miami, Florida, USA: Association for Computational Linguistics, Nov. 2024, pp. 15 838–15 846. [Online]. Available: <https://aclanthology.org/2024.emnlp-main.885/>
- [8] B. Guo, X. Zhang, Z. Wang, M. Jiang, J. Nie, Y. Ding, J. Yue, and Y. Wu, "How close is chatgpt to human experts? comparison corpus, evaluation, and detection," *ArXiv*, vol. abs/2301.07597, 2023. [Online]. Available: <https://api.semanticscholar.org/CorpusID:255998637>
- [9] Y. Liu, Z. Zhang, W. Zhang, S. Yue, X. Zhao, X. Cheng, Y. Zhang, and H. Hu, "Argugpt: evaluating, understanding and identifying argumentative essays generated by gpt models," *ArXiv*, vol. abs/2304.07666, 2023. [Online]. Available: <https://api.semanticscholar.org/CorpusID:258179137>
- [10] Y. Chen, H. Kang, V. J. Zhai, L. Li, R. Singh, and B. Ramakrishnan, "Gpt-sentinel: Distinguishing human and chatgpt generated content," *ArXiv*, vol. abs/2305.07969, 2023. [Online]. Available: <https://api.semanticscholar.org/CorpusID:258686680>
- [11] Z. Wang, J. Cheng, C. Cui, and C. Yu, "Implementing bert and fine-tuned roberta to detect ai generated news by chatgpt," *ArXiv*, vol. abs/2306.07401, 2023. [Online]. Available: <https://api.semanticscholar.org/CorpusID:259145150>
- [12] A. Bhattacharjee, T. Kumarge, R. Moraffah, and H. Liu, "Conda: Contrastive domain adaptation for ai-generated text detection," *ArXiv*, vol. abs/2309.03992, 2023. [Online]. Available: <https://api.semanticscholar.org/CorpusID:261660497>
- [13] R. Koike, M. Kaneko, and N. Okazaki, "Outfox: Llm-generated essay detection through in-context learning with adversarially generated examples," in *AAAI Conference on Artificial Intelligence*, 2023. [Online]. Available: <https://api.semanticscholar.org/CorpusID:260091573>
- [14] K. Krishna, Y. Song, M. Karpinska, J. Wieting, and M. Iyyer, "Paraphrasing evades detectors of ai-generated text, but retrieval is an effective defense," *ArXiv*, vol. abs/2303.13408, 2023. [Online]. Available: <https://api.semanticscholar.org/CorpusID:257687440>
- [15] J. Wu, S. Yang, R. Zhan, Y. Yuan, L. S. Chao, and D. F. Wong, "A survey on LLM-generated text detection: Necessity, methods, and future directions," *Computational Linguistics*, vol. 51, no. 1, pp. 275–338, Mar. 2025. [Online]. Available: <https://aclanthology.org/2025.cl-1.8/>
- [16] G. Bao, Y. Zhao, Z. Teng, L. Yang, and Y. Zhang, "Fast-detectgpt: Efficient zero-shot detection of machine-generated text via conditional probability curvature," *ArXiv*, vol. abs/2310.05130, 2023. [Online]. Available: <https://api.semanticscholar.org/CorpusID:263831345>
- [17] X. Yang, W. Cheng, L. Petzold, W. Y. Wang, and H. Chen, "Dna-gpt: Divergent n-gram analysis for training-free detection of gpt-generated text," *ArXiv*, vol. abs/2305.17359, 2023. [Online]. Available: <https://api.semanticscholar.org/CorpusID:258960101>
- [18] J. Su, T. Y. Zhuo, D. Wang, and P. Nakov, "Detectllm: Leveraging log rank information for zero-shot detection of machine-generated text," *ArXiv*, vol. abs/2306.05540, 2023. [Online]. Available: <https://api.semanticscholar.org/CorpusID:259129463>
- [19] C. Mao, C. Vondrick, H. Wang, and J. Yang, "Raidar: generative ai detection via rewriting," *ArXiv*, vol. abs/2401.12970, 2024. [Online]. Available: <https://api.semanticscholar.org/CorpusID:267095281>
- [20] K. Krishna, Y. Song, M. Karpinska, J. Wieting, and M. Iyyer, "Paraphrasing evades detectors of ai-generated text, but retrieval is an effective defense," in *Proceedings of the 37th International Conference on Neural Information Processing Systems*, ser. NIPS '23. Red Hook, NY, USA: Curran Associates Inc., 2023.
- [21] C. Gu, C. Huang, X. Zheng, K.-W. Chang, and C.-J. Hsieh, "Watermarking pre-trained language models with backdooring," *ArXiv*, vol. abs/2210.07543, 2022. [Online]. Available: <https://api.semanticscholar.org/CorpusID:252907247>
- [22] J. Kirchenbauer, J. Geiping, Y. Wen, M. Shu, K. Saifullah, K. Kong, K. Fernando, A. Saha, M. Goldblum, and T. Goldstein, "On the reliability of watermarks for large language models," *ArXiv*, vol. abs/2306.04634, 2023. [Online]. Available: <https://api.semanticscholar.org/CorpusID:259095643>
- [23] A. Liu, L. Pan, X. Hu, S. Li, L. Wen, I. King, and P. S. Yu, "A private watermark for large language models," *ArXiv*, vol. abs/2307.16230, 2023. [Online]. Available: <https://api.semanticscholar.org/CorpusID:271333036>
- [24] A. B. Hou, J. J. Zhang, T. He, Y. Wang, Y.-S. Chuang, H. Wang, L. Shen, B. V. Durme, D. Khashabi, and Y. Tsvetkov, "Semstamp: A semantic watermark with paraphrastic robustness for text generation," in *North American Chapter of the Association for Computational Linguistics*, 2024. [Online]. Available: <https://api.semanticscholar.org/CorpusID:263831179>